ИТМО · ТЕСТИРОВАНИЕ ПО · ОСЕНЬ 2026

Паттерны API-тестов

Модуль 5 · Лекция 10 · Сценарии, доступ, идемпотентность, асинхронность

Вера Терентьева · Lekton

4 курс

5 ноября · Среда 13:30–15:00



МОДУЛЬ 5 · ЛЕКЦИЯ 10

План лекции

01 Что меняется по сравнению с лекцией 9

02 Позитивный сценарий и два уровня негатива

03 Тестовые данные и изоляция: почему тест падает «раз в пять прогонов»

04 Контроль доступа: 401 против 403, схемы аутентификации, OAuth2

05 Нефункциональные свойства: таймауты, ретраи, лимиты, circuit breaker

06 Идемпотентность: свойство операции, три исхода повторной доставки

07 Асинхронные контракты: RabbitMQ, eventual consistency, Awaitility

08 Интерактивы: покрытие ветвей отказа и ложноположительный тест

МОДУЛЬ 5 · ЛЕКЦИЯ 10

Паттерн — это решение, которое уже кем-то проверено на практике

Вторая лекция модуля не добавляет новых понятий, а превращает словарь HTTP в повторяемые конструкции для типовых ситуаций.

- Курс «Тестирование программного обеспечения», модуль 5 — API-тестирование REST-сервисов.
- Лекция 10: паттерны построения API-тестов — сценарии, доступ, идемпотентность, асинхронность.
- Сегодня вы научитесь различать два уровня негативных сценариев и проектировать тесты на повторную доставку.
- Разберём четыре паттерна и найдём второе расхождение контракта и кода в учебном проекте.
- Практика модуля — артефакт 4: покрытие всех decline-кодов и проверка цепочки сервисов.

МОДУЛЬ 5 · ЛЕКЦИЯ 10

Всё, что мы умеем, проверяет только один запрос

API-тест первого уровня проверяет изолированный запрос, а зрелый тест-сьют проверяет последовательность запросов и эффекты, которые они оставляют в системе.

- Лекция 9 дала: один запрос, один ответ, проверка статуса, тела и заголовков.
- Паттерн начинается там, где появляется второй запрос — проверка состояния после изменения.
- Второй признак паттерна: данные готовятся, используются и изолируются, а не берутся «как есть».
- Третий признак: тест проектируется под конкретный риск, а не «на всякий случай».
- Сегодняшние четыре паттерна покрывают риски: неверный отказ, чужой доступ, дубль, потерю сообщения.

МОДУЛЬ 5 · ЛЕКЦИЯ 10

Сценарии: позитив и два уровня негатива

Позитивный сценарий — эталон, от которого отталкиваются все остальные

> Happy path нужен не для галочки, а чтобы зафиксировать эталонное поведение и увидеть полный набор эффектов одного успешного запроса.

- Позитивный сценарий — валидные данные и корректный бизнес-исход: `200`, `APPROVED`, `responseCode` `00`.
- Проверяем три вещи: транспорт, доменный результат и сгенерированные системой значения.
- Успешная авторизация оставляет эффекты: резервирование средств и запись в очереди на логирование.
- Без зелёного позитивного сценария негативные проверки неинформативны.
- Порядок работы с эндпоинтом всегда одинаков: сначала позитив, потом границы, потом негатив.

У негатива два уровня, и это разные дефекты

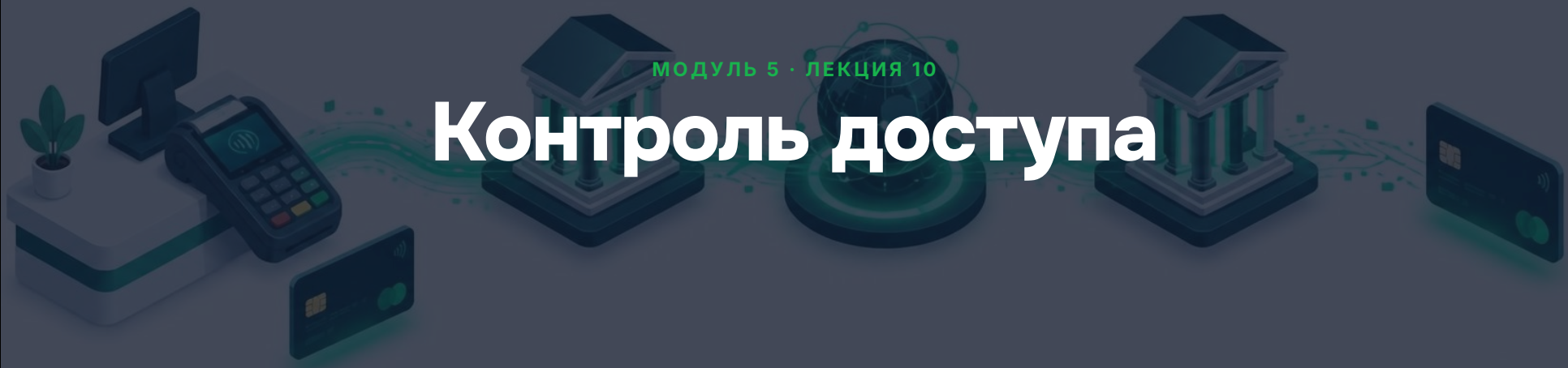
> Ошибка валидации означает, что запрос неверен, а бизнес-отказ означает, что запрос верен, но система приняла отрицательное решение — и эти случаи тестируются по-разному.

- Ошибка валидации: `400`, запрос не соответствует контракту, повтор бессмысленен.
- Бизнес-отказ: запрос корректен, решение отрицательное, это штатное поведение, а не дефект.
- В СМП все бизнес-отказы собраны в перечисление `DeclineOutcome`: пара «причина + ISO-код».
- Внутренний сервис Authorization отвечает `404`, `403`, `422`, `503` по типу причины.
- Клиент через Gateway всегда видит `200` с телом `DECLINED` — Switch гасит внутренний статус.

Тест падает «раз в пять прогонов» — это всегда данные

> Нестабильный API-тест почти всегда означает, что тест зависит от состояния, которое он не контролирует.

- Данные готовятся через API того же сервиса, а не напрямую в базе.
- Каждый тест уникализирует то, что должно быть уникальным: PAN и STAN.
- Уникальный STAN генерируется с привязкой к прогону, а не берётся константой.
- Предыдущие тесты не должны влиять: карта создаётся в `@BeforeEach`, а не переиспользуется.
- Очистка данных — часть теста, если сервис её поддерживает; иначе — уникальные идентификаторы.



МОДУЛЬ 5 · ЛЕКЦИЯ 10

Контроль доступа

Аутентификация отвечает «кто ты», авторизация — «что тебе можно»

Контроль доступа состоит из двух независимых проверок, и тестировать их нужно раздельно, потому что и отказы у них разные.

- Аутентификация подтверждает личность: «это действительно тот, за кого себя выдаёт».
- Авторизация проверяет права: «имеет ли этот субъект доступ к этой операции».
- Основные схемы: API key, Basic Auth, Bearer token по OAuth2.
- В СМП контроля доступа нет: все эндпоинты открыты, это сознательное упрощение.
- Отсутствие доступа в учебном проекте — не повод не уметь его тестировать.

Четыре теста на доступ закрывают весь класс эквивалентности

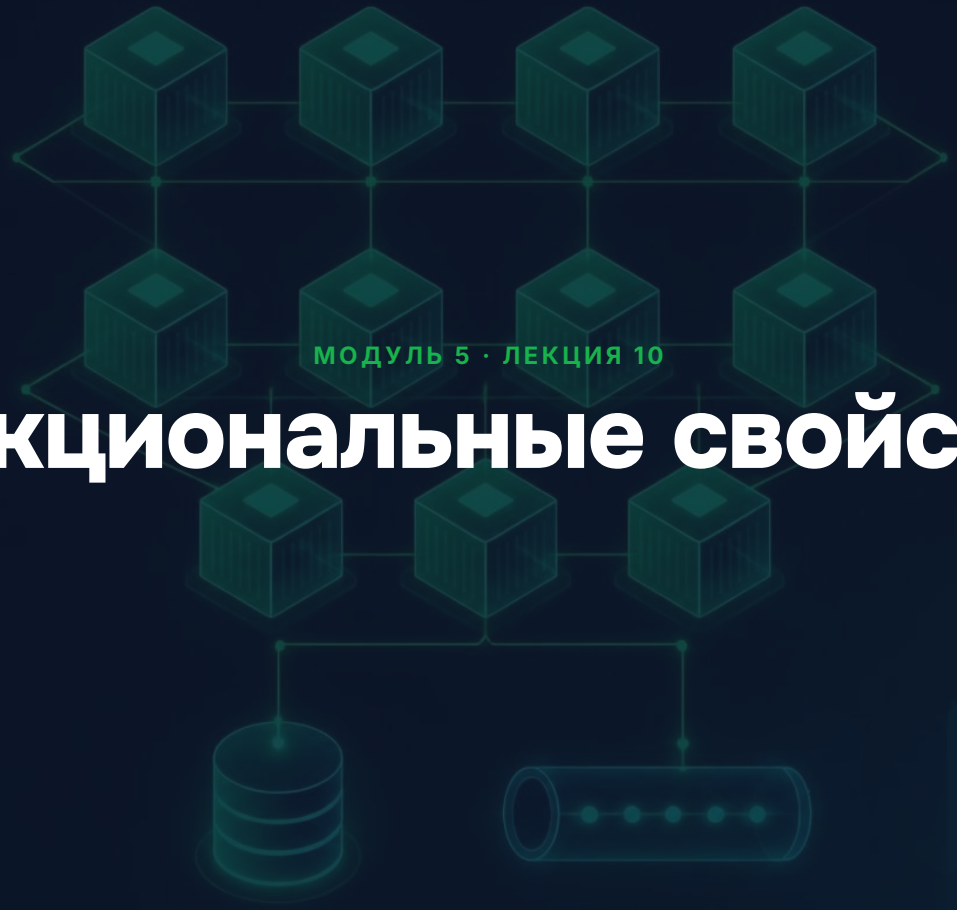
Контроль доступа тестируется четырьмя классами запросов, и каждый класс даёт предсказуемый статус-код.

- Без токена — `401`: сервер обязан отказать, а не вернуть данные.
- С истёкшим или поддельным токеном — `401`: различать эти случаи по статусу не нужно.
- С валидным токеном без нужных прав — `403`: субъект опознан, операция запрещена.
- С валидным токеном и правами — `200`: доступ разрешён.
- В REST-assured токен задаётся один раз в `RequestSpecification`, а не в каждом тесте.
- Схема применима к реальным проектам: в СМП доступа нет, поэтому четыре теста здесь писать не к чему.

OAuth2 — это про срок жизни и права, а не про «просто пароль»

> Токен OAuth2 имеет два измеримых свойства — срок жизни и набор прав, — и оба нужно тестировать на границах.

- Поток: клиент запрашивает токен у сервера авторизации и шлёт его с каждым запросом.
- `expires_in` задаёт срок жизни; после истечения сервер обязан вернуть `401`.
- `scope` задаёт набор прав; запрос вне скоупа должен вернуть `403`.
- Refresh token позволяет получить новый access-токен без повторного входа.
- Тестируем границу срока жизни, а не только факт истечения.



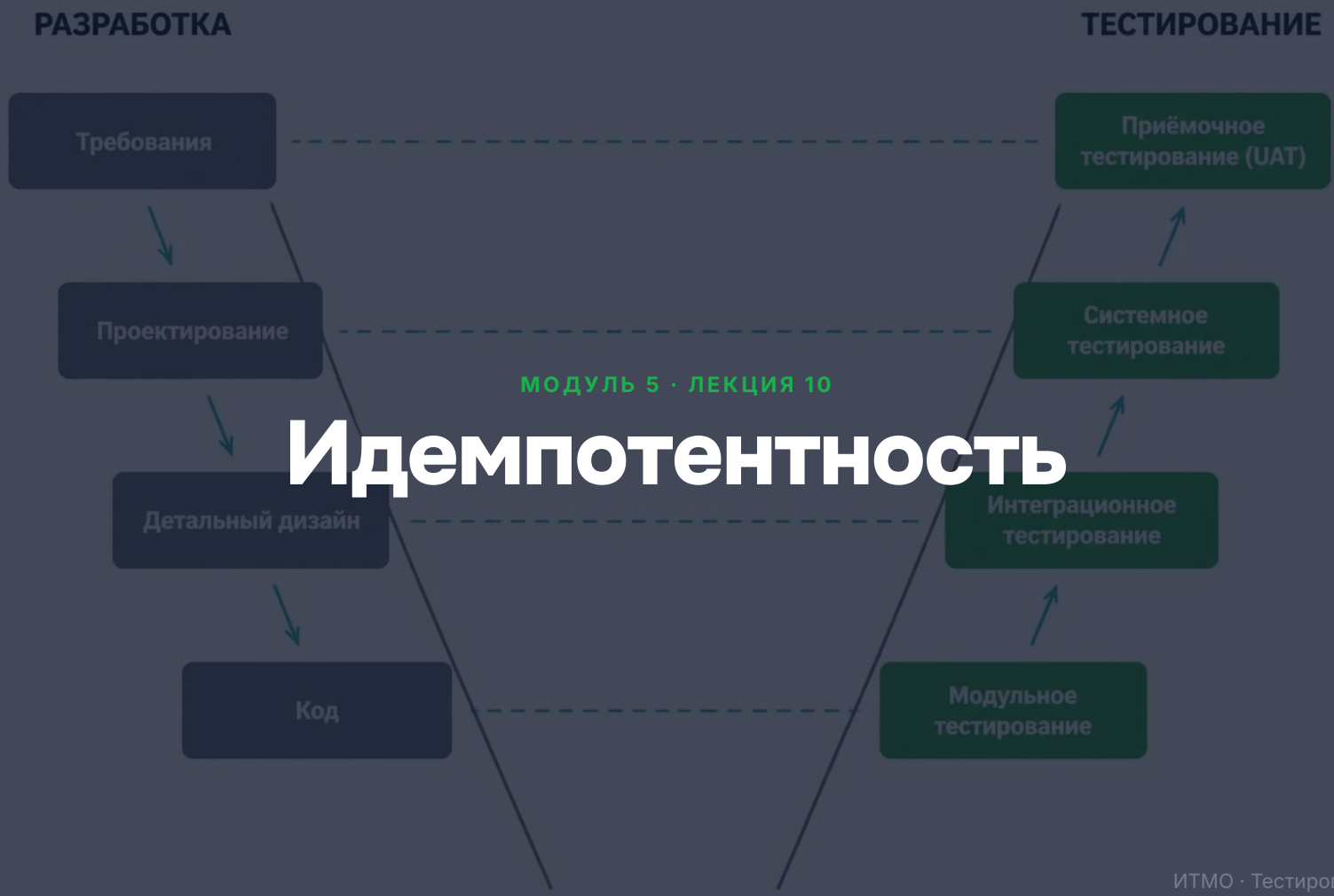
МОДУЛЬ 5 · ЛЕКЦИЯ 10

Нефункциональные свойства API

У API есть нефункциональные свойства, и они ломаются первыми

Таймауты, повторные попытки, ограничения частоты и размыкание цепи — это контракт поведения при сбоях, и он тестируется так же, как формат ответа.

- Gateway ограничивает частоту запросов на авторизацию: сто в секунду на IP, при превышении `429`.
- Switch повторяет вызов Authorization до трёх раз с паузами 1, 2 и 4 секунды.
- Таймауты: соединение 3 секунды, чтение ответа Authorization 5 секунд, публикация в очередь 2 секунды.
- Circuit breaker в Switch размыкается при 50 процентах ошибок в окне из 10 вызовов на 30 секунд.
- Каждое из этих значений — проверяемое поведение, а не «настройка инфраструктуры».



МОДУЛЬ 5 · ЛЕКЦИЯ 10

Идемпотентность

Идемпотентность — свойство операции, а не сервера

> Операция идемпотентна, если повторная её отправка с теми же данными не создаёт дополнительных эффектов.

- Идемпотентность — это про эффекты, а не про код ответа: повтор может вернуть другой статус при том же итоге.
- `GET`, `PUT` и `DELETE` идемпотентны по семантике HTTP, `POST` — нет по умолчанию.
- В финансовых системах идемпотентность — требование безопасности, а не удобства.
- В СМП идемпотентность реализована в Transaction Logger: дедупликация по `id`, который генерирует Switch.
- В пути авторизации дедупликации по STAN нет — и это отдельный риск, который стоит зафиксировать.

Повторная доставка — не редкий случай, а штатный режим работы

> В распределённой системе клиент не может отличить «запрос не дошёл» от «запрос дошёл, ответ потерялся», и поэтому повторы неизбежны.

- Сетевые сбои, таймауты и автоматические ретраи создают повторные запросы без участия человека.
- Брокер сообщений доставляет по семантике «хотя бы один раз»: дубли возможны по контракту.
- Ключ дедупликации должен идентифицировать именно запрос, а не карту или сумму.
- В СМП ключ — UUID, который генерирует Switch один раз при публикации.
- Три исхода повторной публикации в Logger: новая запись, существующая запись, конфликт данных.

Тест идемпотентности проверяет три исхода, а не один

> Тест идемпотентности должен различать повтор с теми же данными и повтор с расходящимися данными, иначе он проверяет только половину контракта.

- Публикуем транзакцию напрямую в Logger на порт 8088: маршрута `/api/internal/log` через Gateway нет.
- Первая публикация — `201`, повтор с тем же телом — `200`, повтор с другим `amount` — `409`.
- Проверка «запись ровно одна» делается поиском по рабочим фильтрам и отбором по `id` на стороне теста.
- Фильтра `stan` в поиске нет — использовать его в тесте нельзя, иначе тест ничего не проверяет.
- Отсутствие дедупликации в пути авторизации проверяется отдельным исследовательским тестом.

Код теста идемпотентности: два запроса и GPath-фильтр (1/2)

Три исхода повторной доставки проверяются тремя утверждениями: `201` на первой публикации, `200` на повторе и ровно одна запись по нашему идентификатору.

```
@Test
void shouldNotDuplicateTransactionOnRedelivery() {
    String id = UUID.randomUUID().toString();
    String pan = createCard();          // карта под этот тест
    TransactionRequest tx = transactionWithId(id, pan, "000123", 150000L);

    // Первая публикация — новая запись
    given()
        .baseUri("http://localhost:8088")
        .contentType(ContentType.JSON)
        .body(tx)
    .when()
        .post("/api/internal/log")
    .then()
        .statusCode(201);

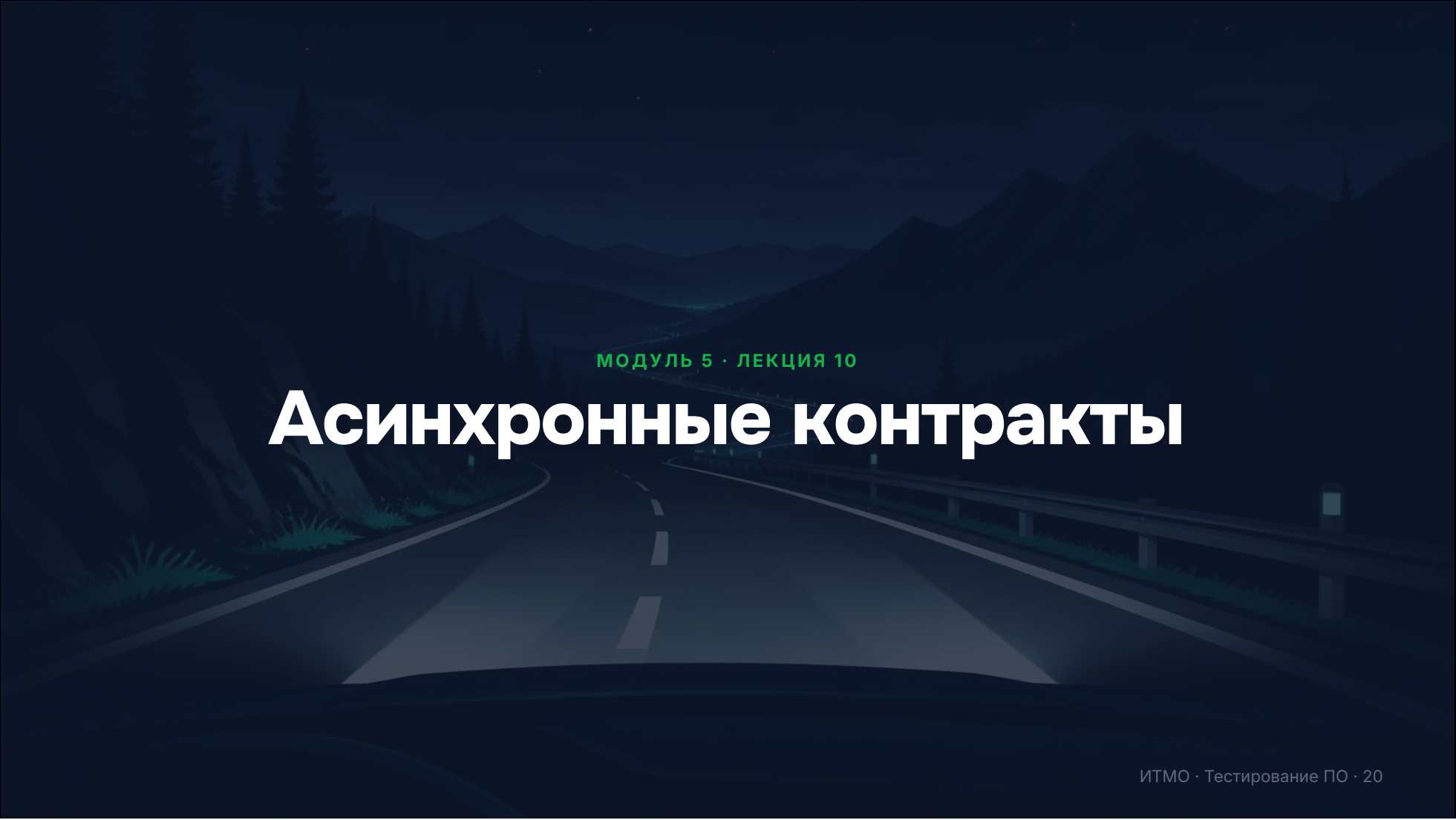
    // Повторная доставка того же сообщения — запись уже есть
    given()
        .baseUri("http://localhost:8088")
        .contentType(ContentType.JSON)
        .body(tx)
    .when()
        .post("/api/internal/log")
    .then()
        .statusCode(200);

    // В базе должна быть ровно одна запись с нашим id
    given()
        .baseUri("http://localhost:8080")
        .queryParam("pan", pan)
        .queryParam("dateFrom", "2026-06-01")
    .when()
        .get("/api/transactions/search")
    .then()
        .statusCode(200)
```

Код теста идемпотентности: два запроса и GPath-фильтр (2/2)

```
        .body("transactions.findAll { it.id == '" + id + "' }.size()", equalTo(1));
}
```

- Порт 8088 для публикации, 8080 для поиска — это два разных контракта.
- `201` и `200` различают «создано» и «уже было»: это и есть проверка дедупликации.
- GPath-фильтр по `id` заменяет отсутствующий параметр `stan`.



МОДУЛЬ 5 · ЛЕКЦИЯ 10

Асинхронные контракты

Асинхронный контракт — это тоже контракт, только ответ приходит отдельно

При публикации в очередь контракт описывает не только формат сообщения, но и гарантии доставки, порядок и поведение при отказах.

- Синхронный контракт: запрос и ответ в одном вызове; асинхронный — публикация, доставка, потребление.
- В СМП Switch публикует в обменник `smp.transactions` с ключом маршрутизации `transaction.log`.
- Logger слушает очередь `transaction-log`, сохраняет в PostgreSQL и рассылает по WebSocket.
- Очередь настроена с обменником мёртвых писем и лимитом в три доставки, затем сообщение уходит в `transaction-log-dlq`.
- У сообщений в очереди мёртвых писем TTL шестьдесят секунд — для диагностики, не для хранения.

Согласованность в конечном счёте ломает наивные проверки

> Если эффект асинхронный, тест обязан ждать условие, а не время.

- Согласованность в конечном счёте: система придёт в согласованное состояние, но не мгновенно.
- Пауза фиксированной длительности хрупка: медленно под нагрузкой и расточительна в норме.
- Awaitility опрашивает условие с заданным интервалом до истечения предельного срока.
- Проверять надо условие предметной области, а не факт «прошло сколько-то времени».
- Реальные параметры в тестах СМП: до 15 секунд, первая проверка через 500 мс, далее раз в секунду.

В СМП три вида контрактов, и у каждого свой способ проверки

Синхронный HTTP, очередь сообщений и WebSocket проверяются разными инструментами и разными ожиданиями.

- HTTP: ответ сразу, проверяется статус, тело и заголовки; ожидание не требуется.
- Очередь: ответа нет, проверяется появление записи через вторичный запрос с ожиданием.
- WebSocket: поток сообщений, проверяется получение конкретного события в отведённое время.
- Для HTTP годится REST-assured, для очереди — Awaitility плюс поиск, для WebSocket — клиент и таймаут.
- Главный риск у всех трёх общий: эффект есть, но потребитель его не видит.

МОДУЛЬ 5 · ЛЕКЦИЯ 10

Асинхронный тест и пагинация

МОДУЛЬ 5 · ЛЕКЦИЯ 10

Тест асинхронного контракта: ждём эффект и проверяем режимы отказа

> Позитивный асинхронный тест — это ожидание эффекта по наблюдаемому признаку; негативный — проверка, что эффекта нет и состояние не испорчено.

- Отправляем транзакцию через Gateway, затем ждём появления записи поиском по `pan` и `dateFrom`.
- Нужную запись отбираем по `stan` внутри ответа через GPath, потому что фильтра `stan` нет.
- Режимы отказа: недоступность брокера, исчерпание попыток доставки, повторная доставка.
- При недоступности брокера успешная авторизация откатывается и клиент получает код `96`.
- Проверка отсутствия эффекта требует ожидания «наоборот»: ждём срок и убеждаемся, что записи нет.

Код асинхронного теста: ожидание условия вместо паузы

Awaitility превращает «подождать непонятно сколько» в проверяемое условие с явными границами по времени.

```
@Test
void shouldEventuallyConsumeTransaction() {
    String pan  = createCard();                 // карта под этот тест
    String stan = uniqueStan();                 // уникальный STAN на прогон

    // Отправляем транзакцию через Gateway
    given()
        .baseUri("http://localhost:8080")
        .contentType(ContentType.JSON)
        .body(transactionJson(pan, stan, 150000L))
    .when()
        .post("/api/transactions")
    .then()
        .statusCode(200)
        .body("status", equalTo("APPROVED"));

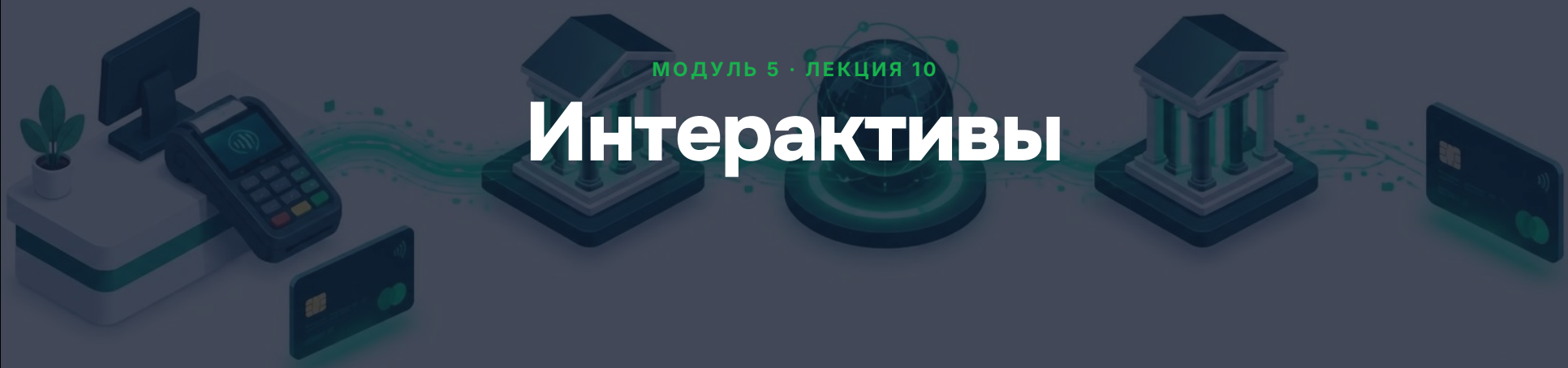
    // Ждём появления записи в Logger: поиск по рабочим фильтрам,
    // отбор нужной записи — по stan внутри массива
    await()
        .atMost(Duration.ofSeconds(15))
        .pollDelay(Duration.ofMillis(500))
        .pollInterval(Duration.ofSeconds(1))
        .untilAsserted(() → given()
            .baseUri("http://localhost:8080")
            .queryParam("pan", pan)
            .queryParam("dateFrom", "2026-06-01")
        .when()
            .get("/api/transactions/search")
        .then()
            .statusCode(200)
            .body("transactions.findAll { it.stan == '" + stan + "' }.size()",
                  greaterThanOrEqualTo(1)));
}
```

- `atMost` — предел ожидания, `pollInterval` — шаг опроса, `pollDelay` — первая пауза.
- Внутри `untilAsserted` — обычные проверки REST-assured, а не ручное сравнение строк.
- Ищем по `pan` и `dateFrom`, а нужную запись отбираем GPath-фильтром по `stan`.

Пагинация и фильтры: проверяем границы и комбинации

Пагинация и фильтры — самая частая область дефектов, потому что здесь пересекаются валидация, сортировка и полнотекстовый поиск.

- `limit` валиден от 1 до 500, значение по умолчанию 50; `offset` — неотрицательный, по умолчанию 0.
- `limit = 0`, отрицательный `offset` и `limit = 501` дают `400`.
- Фильтры поиска: `pan`, `status`, `dateFrom`, `dateTo`, `merchantId`, `issuerId`, `mcc`, `declineReason`.
- `declineReason` — полнотекстовый поиск, а не точное совпадение.
- Пустой результат — это `200` с `total = 0`, а не `404`.



МОДУЛЬ 5 · ЛЕКЦИЯ 10

Интерактивы

Интерактив: спроектируйте тест-кейсы для эндпоинта авторизации

- Дано: эндпоинт авторизации, одиннадцать исходов в перечислении `DeclineOutcome`.
- Вопрос первый: сколько минимум тестов нужно, чтобы покрыть все ветки отказа?
- Вопрос второй: какие ветки делят один код и требуют проверки причины?
- Вопрос третий: как проверить, что при недоступности брокера транзакция не осталась одобренной?
- На обсуждение три минуты, отвечаем по одному вопросу от группы.

Интерактив: зелёный тест, который ничего не проверяет

- Тест проверяет идемпотентность: два одинаковых запроса, затем поиск результата по `stan`.
- Ожидание: в результате ровно одна запись — `total` равен единице.
- Вопрос: почему тест зелёный, что именно он проверил и как его исправить?

Выводы и анонс

> Паттерн API-теста — это осознанный ответ на конкретный риск, а не набор проверок «на всякий случай».

- Негатив двух уровней: `400` на нарушение контракта и бизнес-отказ со `200` и `DECLINED` в теле.
- Внутренний сервис отдаёт `404`, `403`, `422`, `503`, а клиент через Gateway видит только `200`.
- Контроль доступа проверяется четырьмя классами запросов: без токена, с плохим, без прав, с правами.
- Идемпотентность в СМП есть у Logger и отсутствует в пути авторизации — это находка для баг-репорта.
- Асинхронные контракты проверяются ожиданием условия, а не паузой; отсутствие эффекта ждут отдельно.
- Следующий модуль: интеграционное тестирование с базой данных в контейнерах.

Что унести с лекции

- Негатив двух уровней: `400` на нарушение контракта и бизнес-отказ со `200` и `DECLINED`
- Внутренний сервис отдаёт `404`, `403`, `422`, `503` — клиент через Gateway видит только `200`
- Контроль доступа: четыре класса запросов плюс две границы токена
- Идемпотентность есть у Logger и отсутствует в пути авторизации — это находка
- Асинхронные контракты проверяются ожиданием условия, а не паузой
- Ложноположительный тест опаснее отсутствующего
- Следующий модуль: интеграционное тестирование с базой в контейнерах

Источники

- **Spillner, A., Linz, T., & Schaefer, H.** (2014). *Software Testing Foundations* (4th ed.). Rocky Nook. ISBN 978-1-937538-42-2. — Глава 3.3 (интеграционное тестирование), 3.7.2 (нефункциональное тестирование).
- **Newman, S.** (2021). *Building Microservices* (2nd ed.). O'Reilly. ISBN 978-1-492-03402-5. — Глава 4 (согласованность в конечном счёте), Глава 9 (тестирование).
- **Myers, G., Sandler, C., & Badgett, T.** (2011). *The Art of Software Testing* (3rd ed.). Wiley. ISBN 978-1-118-03196-4. — Глава 4 (Test-Case Design).
- **Khorikov, V.** (2020). *Unit Testing: Principles, Practices, and Patterns*. Manning. ISBN 978-1-61729-627-7. — мутационная проверка качества тестов.
- OAuth 2.0 specification: https://oauth.net/2/
- **RFC 9110**: *HTTP Semantics*, раздел об идемпотентности методов. https://www.rfc-editor.org/rfc/rfc9110.html
- REST-assured documentation: https://rest-assured.io/
- Awaitility: https://github.com/awaitility/awaitility
- RabbitMQ documentation, разделы о подтверждениях публикации и обменниках мёртвых писем: https://www.rabbitmq.com/docs
- Resilience4j — circuit breaker и retry: https://resilience4j.readme.io/
- **Спецификация и код СМП**: `docs/api-spec.md`, `services/authorization`, `services/transaction-logger`, `services/switch` — источники примеров.

Вопросы?

Паттерны API-тестов · модуль 5

